

# Testeo TechMind

Federico G. Gutierrez  
**Tester QA**

VERSIÓN 2.0

---

# CONTENIDOS

- 1** Informe de resultados
- 2** Detalle de casos
- 3** Próximas tareas

# Informe de resultados

---

**Proyecto: TechMind - Organización Inteligente del Conocimiento Técnico**

Responsable QA: Federico G. Gutierrez

Fecha de Ejecución: 28 de Julio de 2026

## Resumen Ejecutivo

Durante el Sprint 1 y su fase de Data QA, se ejecutó la suite completa de pruebas y auditoría sobre los endpoints de la API FastAPI y la base de datos PostgreSQL (**techmind**), evaluando la persistencia y calidad de datos en las tablas contenidos y predicciones. Con ello se logró una cobertura total de escenarios funcionales, de límites (superiores e inferiores), seguridad, resiliencia (ante inyección de datos), validación, integridad referencial, consistencia de esquemas y unicidad de claves primarias.

## Métricas Generales

- Casos Planificados: 41
- Casos Ejecutados: 41
- Casos Exitosos: 41
- Casos Fallidos: --
- **Porcentaje de Éxito: 100 %**

Desglose por Tipo de Prueba (FastAPI)

| Categoría                                | Planificados | PASÓ | FALLÓ | % Éxito |
|------------------------------------------|--------------|------|-------|---------|
| Funcionales (Flujo Feliz)                | 8            | 8    | --    | 100     |
| Casos Borde / Encoding (UTF-8)           | 3            | 3    | --    | 100     |
| Validación de Esquema / Tipos            | 3            | 3    | --    | 100     |
| Seguridad (Inyección SQL / Content-Type) | 2            | 2    | --    | 100     |
| Endpoints Complementarios                | 2            | 2    | --    | 100     |
| Integridad de Datos                      | 6            | 6    | --    | 100     |

Conclusión y Recomendaciones

El MVP cumple con todos los criterios de aceptación funcionales, de rendimiento (< 2000 ms) y de seguridad especificados para el Sprint 1. La API demuestra alta estabilidad y resiliencia ante errores de infraestructura.

**Desglose por Tipo de Prueba (PostgreSQL - techmind)**

| Categoría                      | Planificados | PASÓ | FALLÓ | % Éxito |
|--------------------------------|--------------|------|-------|---------|
| Compleitud                     | 5            | 5    | --    | 100     |
| Integridad y Estructura        | 6            | 6    | --    | 100     |
| Formato y Calidad de Texto     | 3            | 3    | --    | 100     |
| Edge Cases (Límites y Vacíos)  | 2            | 2    | --    | 100     |
| Rendimiento e Inserción Masiva | 1            | 1    | --    | 100     |

**Conclusión y Recomendaciones**

La capa de datos en PostgreSQL demostró total solidez técnica en el Sprint 1 al garantizar la integridad de los datos (claves primarias, campos obligatorios y unicidad), alta resiliencia y seguridad (protección contra SQLi y soporte de estructuras complejas/TEXT) y excelente escalabilidad frente a inserciones masivas y grandes volúmenes de datos.



# Detalle de casos

---

## Testeo Funcionales (Flujo Feliz) — [8 casos]

- **CP-FASTAPI-01:** Valida que la API clasifique correctamente un contenido técnico válido devuelto con HTTP 200.
- **CP-FASTAPI-02:** Comprueba la extracción y retorno correcto de la lista de palabras clave relevantes en la respuesta.
- **CP-FASTAPI-03:** Verifica la generación de un score de probabilidad de clasificación válido entre 0 y 1.
- **CP-FASTAPI-09:** Confirma que la estructura del objeto JSON devuelto contenga todas las claves y tipos de datos especificados.
- **CP-FASTAPI-10:** Valida la correcta inicialización y carga de los modelos serializados (.joblib) al arrancar el servicio.
- **CP-FASTAPI-11:** Mide el tiempo de respuesta del endpoint asegurando que la inferencia se ejecute en menos de 2000 ms.
- **CP-FASTAPI-20:** Verifica que el modelo normalice y procese adecuadamente textos ingresados con tildes y mayúsculas sostenidas.
- **CP-FASTAPI-22:** Evalúa la latencia, estabilidad y cero tasa de errores del endpoint ante 100 peticiones concurrentes en ráfaga.

## Casos Borde / Encoding (UTF-8) — [3 casos]

- **CP-FASTAPI-12:** Evalúa el comportamiento y la gestión de memoria de la API ante payloads de gran tamaño (+500k caracteres).
- **CP-FASTAPI-13:** Garantiza la sanitización y el soporte de codificación UTF-8 procesando caracteres especiales, etiquetas y emojis sin errores.
- **CP-FASTAPI-21:** Evalúa cómo reacciona la API ante entradas no técnicas o sin sentido.

## Validación de Esquema / Tipos — [3 caso]

- **CP-FASTAPI-14:** Confirma el rechazo inmediato (HTTP 422) por parte de Pydantic al enviar tipos de datos no válidos (números o booleanos).
- **CP-FASTAPI-23:** Valida que la API rechace payloads nulos devolviendo 422 Unprocessable Entity cuando el cuerpo enviado está vacío.
- **CP-FASTAPI-24:** Valida que la API detecte errores de sintaxis JSON y rechace la petición con un 422 Unprocessable Entity.

## Seguridad (Inyección SQL / Content-Type) — [2 casos]

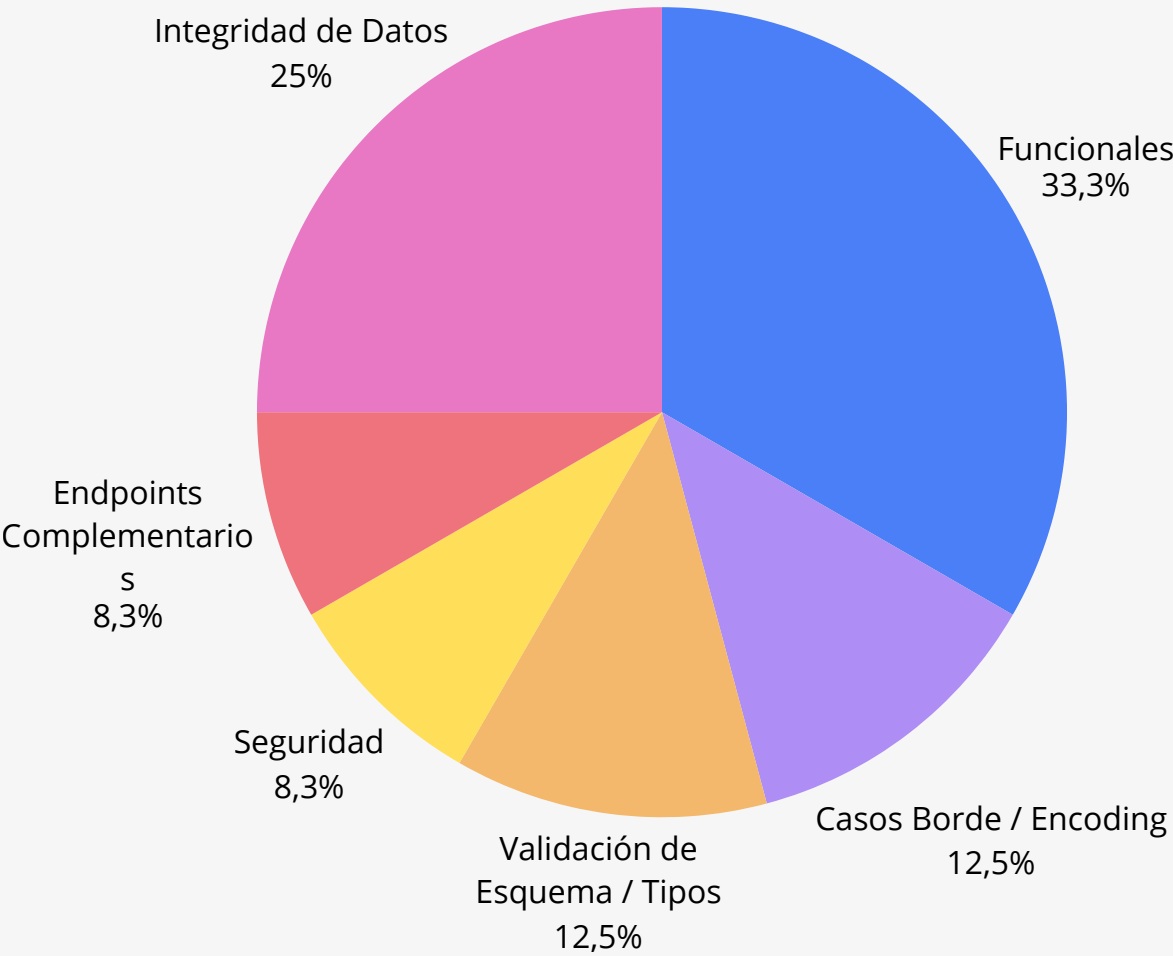
- **CP-FASTAPI-15:** Comprueba la inmunidad ante intentos de inyección SQL guardando los comandos maliciosos como texto plano.
- **CP-FASTAPI-16:** Valida el rechazo de solicitudes con formatos no soportados como XML mediante una respuesta HTTP 422.

### Endpoints Complementarios — [2 casos]

- **CP-FASTAPI-17:** Verifica la disponibilidad y estado operativo del microservicio mediante el endpoint de diagnóstico GET /health.
- **CP-FASTAPI-18:** Valida que el endpoint GET /categorias retorne el catálogo completo con las 8 categorías del modelo.

### Integridad de Datos — [6 casos]

- **CP-FASTAPI-04:** Controla el rechazo de la solicitud (HTTP 422) cuando el campo "titulo" se envía vacío.
- **CP-FASTAPI-05:** Controla el rechazo de la solicitud (HTTP 422) cuando el campo "texto" no contiene información.
- **CP-FASTAPI-06:** Valida la respuesta de error adecuada al enviar tanto el título como el texto vacíos en el payload.
- **CP-FASTAPI-07:** Comprueba la interceptación y manejo de excepciones ante la recepción de una estructura JSON mal formada.
- **CP-FASTAPI-08:** Verifica el rechazo con código HTTP 405 Method Not Allowed al enviar una petición GET al endpoint POST /predecir.
- **CP-FASTAPI-19:** Verifica que la API rechace la petición cuando se omiten claves obligatorias dentro del cuerpo JSON.



---

## Completitud — [5 casos]

- **CP-DB-01:** Verifica la ausencia de filas incompletas en contenidos.
- **CP-DB-02:** Confirma de que no existen registros de texto idénticos sobrecargando la base de datos.
- **CP-DB-03:** Distribución uniforme y validación sobre las 8 categorías temáticas del modelo.
- **CP-DB-04:** Coincidencia del 100% entre la respuesta JSON de la API y el registro persistido en predicciones.
- **CP-DB-05:** Aprobación de rango numérico estrictamente acotado en [0,0; 1,0].

## Integridad y Estructura — [6 casos]

- **CP-DB-08:** 100% de unicidad en la columna id de predicciones con secuencia autoincremental coherente.
- **CP-DB-09:** Fechas de la columna created\_at válidas, no nulas y alineadas a la zona horaria actual.
- **CP-DB-10:** Confirmación de cadenas válidas y limpias sin presencia de nulos ni espacios aislados.
- **CP-DB-11:** Categorías predichas pertenecientes de forma estricta al dominio semántico (Backend, Bases de Datos, etc.).
- **CP-DB-13:** Coincidencia perfecta del 100% en campos clave de la tabla de origen contenidos.
- **CP-DB-14:** Verifica las restricciones PRIMARY KEY sobre la tabla contenidos.

## Formato y Calidad de Texto — [3 caso]

- **CP-DB-06:** Valida el tipo de dato text[] para palabras clave e inferencias con arreglos vacíos {}.
- **CP-DB-07:** Sanitización e inserción exitosa del texto literal conteniendo comandos destructivos (ej. '; DROP TABLE...') sin alteración de la estructura.
- **CP-DB-15:** Evaluación cualitativa de textos cortos garantizando significancia técnica para el modelo NLP.

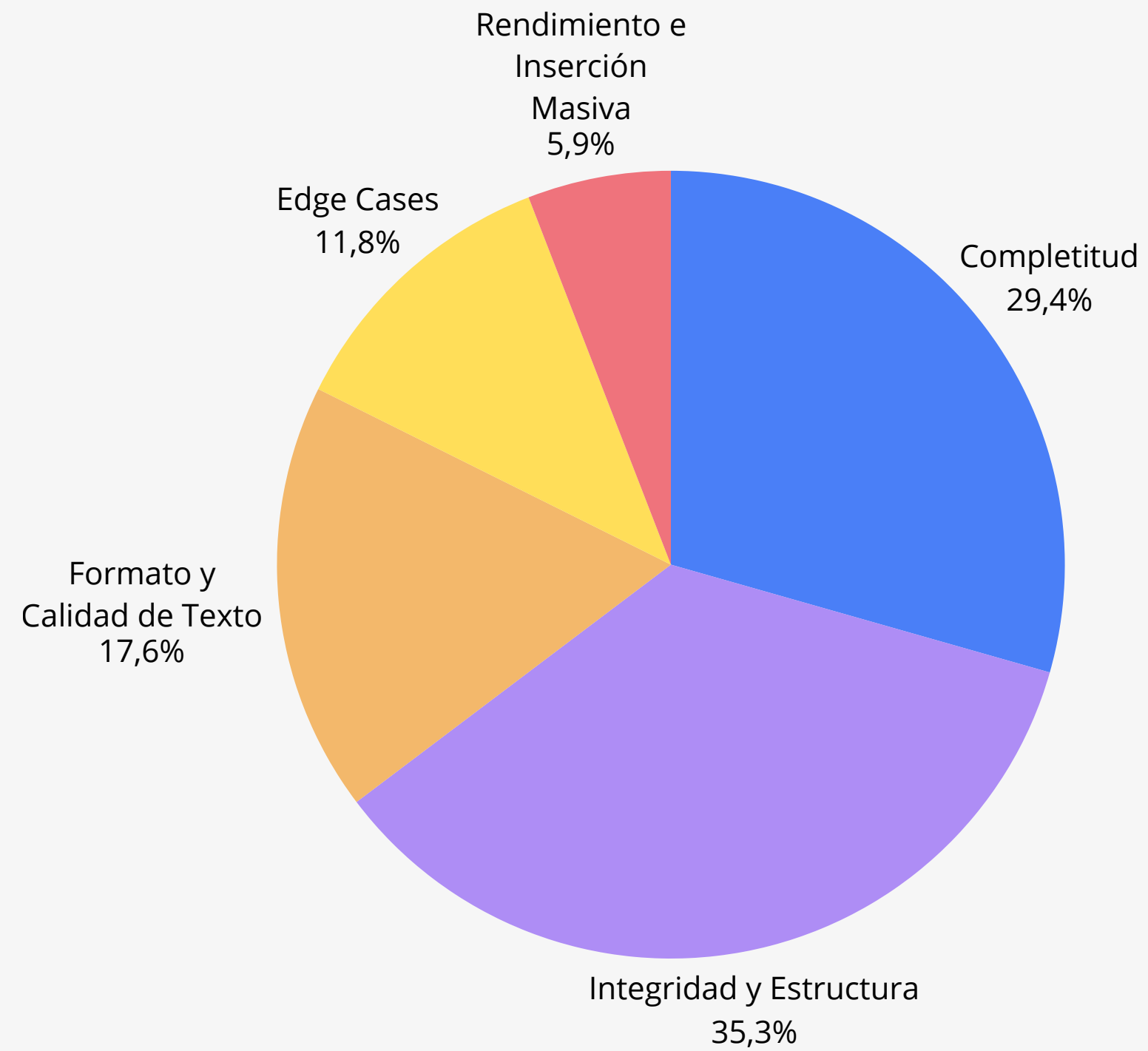
## Edge Cases (Límites y Vacíos) — [2 casos]

- **CP-DB-16:** Validación de rechazo/ausencia de textos con longitud 0 o TRIM nulo.
- **CP-DB-17:** Confirmación de persistencia íntegra de payloads extensos en el tipo de dato TEXT sin truncamiento.

## Rendimiento e inserción masiva — [1 caso]

- **CP-DB-12:** Resistencia y persistencia exitosa ante ráfagas de carga masiva de hasta 300 peticiones/minuto.





**Matriz**

**Evidencias**

# Próximas tareas

---

Para continuar fortaleciendo la calidad y madurez del proyecto, se proponen las siguientes iniciativas para la próxima etapa:

- **Automatización en Pipeline CI/CD (GitHub Actions + Newman):** Integrar la ejecución automática de la suite de pruebas de Postman en cada despliegue o pull request, asegurando la detección temprana de regresiones antes de pasar a entornos de producción.
- **Monitoreo Continuo y Alertas en Producción (OCI):** Configurar un esquema de lectura centralizada de logs en Oracle Cloud Infrastructure para alertar de forma automática ante caídas de servicio, errores 500 o fallos de conexión a la base de datos.